

Optimization Methods

A Beginner's Companion Guide

Mini-batch gradient descent, Momentum, and Adam — the components, the formulas, and why they work

About This Guide

Making the Same Network Learn Faster

A plain-language walkthrough of the three optimization methods — and the choices behind each one.

Once a network trains correctly, the next question is how to make it train *quickly*. Applying deep learning is a deeply empirical, iterative process: you train many models to find one that works well, so anything that speeds up each training run speeds up your whole project. This guide walks through the three optimization methods you implemented in the assignment, in the order they build on one another — mini-batch gradient descent, then Momentum, then Adam — followed by learning rate decay, the tuning trick that helps any of them settle into a good answer.

Every method here answers the same question in a smarter way: *given the gradient, in which direction and how far should I step the weights?* Plain gradient descent uses only the current gradient. Momentum averages the recent gradients to cancel out noise. Adam goes further, also scaling each parameter's step by how large its gradients have been. Understanding that progression is the key to the whole topic.

HOW TO READ THIS GUIDE

Each section follows the same rhythm: the idea in everyday language, the code that implements it, and a short note on where it fits. You need no mathematics beyond multiplication, addition, and the idea of an average. The companion script `optimization_methods_teaching_demo.py` contains every function shown here, ready to run, and trains one tiny model with all three optimizers so you can watch the differences yourself.

1. Mini-Batch Gradient Descent: Progress Before You Finish the Data

Deep learning works best with lots of data, but lots of data is slow. With plain (“batch”) gradient descent, you must process the *entire* training set — perhaps five million examples — before taking a single step. That is wasteful. The fix is to let the algorithm make progress before it has read everything.

KEY CONCEPT: THE MINI-BATCH

A **mini-batch** is a small slice of the training set — commonly 64, 128, or 256 examples. Instead of one weight update per full pass over the data, you take one update per mini-batch, so a single pass (an **epoch**) yields many updates. **Batch** gradient descent uses the whole set at once (one update per epoch); **stochastic** gradient descent uses one example at a time (very noisy); mini-batch sits in between and is what almost everyone uses in practice.

Building the mini-batches takes two steps: shuffle the data so each epoch sees a fresh order, then slice it into chunks. The last chunk may be smaller than the rest, and that is fine.

```
def random_mini_batches(X, Y, mini_batch_size=64, seed=0):
    m = X.shape[1]
    mini_batches = []

    # Step 1: shuffle the columns (examples) of X and Y the SAME way
    permutation = list(np.random.permutation(m))
    shuffled_X = X[:, permutation]
    shuffled_Y = Y[:, permutation].reshape((1, m))

    # Step 2: carve out the full-size batches
    num_complete = math.floor(m / mini_batch_size)
    for k in range(num_complete):
        mb_X = shuffled_X[:, k*mini_batch_size:(k+1)*mini_batch_size]
        mb_Y = shuffled_Y[:, k*mini_batch_size:(k+1)*mini_batch_size]
        mini_batches.append((mb_X, mb_Y))

    # Step 3: the leftover examples form one smaller final batch
    if m % mini_batch_size != 0:
        mb_X = shuffled_X[:, num_complete*mini_batch_size:]
        mb_Y = shuffled_Y[:, num_complete*mini_batch_size:]
        mini_batches.append((mb_X, mb_Y))

    return mini_batches
```

The update itself is the ordinary gradient descent rule — step each parameter directly against its gradient — just applied once per mini-batch rather than once per epoch:

```
def update_parameters_with_gd(parameters, grads, learning_rate):
    L = len(parameters) // 2
    for l in range(1, L + 1):
        parameters["w" + str(l)] -= learning_rate * grads["dw" + str(l)]
        parameters["b" + str(l)] -= learning_rate * grads["db" + str(l)]
    return parameters
```

Because each step is based on only a slice of the data, the cost no longer falls perfectly smoothly — it wobbles down rather than gliding down. That noise is a feature, not a bug: you make many more steps in the same wall-clock time, so you usually reach a good answer far sooner.

2. The Building Block: Exponentially Weighted Averages

Before Momentum and Adam make sense, you need one idea they both rely on: the exponentially weighted average. It is a way of keeping a running, smoothed value that leans on its own recent history.

KEY CONCEPT: THE EXPONENTIALLY WEIGHTED AVERAGE

The recursion is $v = \text{beta} * v + (1 - \text{beta}) * \text{new_value}$. The number beta (between 0 and 1) sets how much you trust the accumulated past versus the latest value. A larger beta averages over

more history and gives a smoother, slower-moving result; roughly, it averages the last $1 / (1 - \text{beta})$ values. With $\text{beta} = 0.9$, that is about the last ten.

The reason this matters for optimization: gradients computed on mini-batches are noisy. If you average them this way, the noise — which points in random directions and tends to cancel — smooths out, while the consistent downhill direction survives. That smoothed gradient is exactly what Momentum steps along.

3. Momentum: Smooth the Gradients, Then Step

Imagine a cost surface shaped like a long, narrow valley. Plain gradient descent zig-zags across the steep walls while only slowly creeping along the valley floor toward the minimum. Momentum fixes this by stepping along an *average* of the recent gradients rather than the latest one. The side-to-side wobbles average out to nearly nothing; the steady forward direction adds up.

First, set up the running average — called the **velocity** — as zeros, one array per parameter:

```
def initialize_velocity(parameters):
    L = len(parameters) // 2
    v = {}
    for l in range(1, L + 1):
        v["dw" + str(l)] = np.zeros(parameters["w" + str(l)].shape)
        v["db" + str(l)] = np.zeros(parameters["b" + str(l)].shape)
    return v
```

Then, on each step, update the velocity with the new gradient and step along it:

```
def update_parameters_with_momentum(parameters, grads, v, beta, learning_rate):
    L = len(parameters) // 2
    for l in range(1, L + 1):
        # velocity = smoothed gradient (the exponentially weighted average)
        v["dw" + str(l)] = beta*v["dw" + str(l)] + (1-beta)*grads["dw" + str(l)]
        v["db" + str(l)] = beta*v["db" + str(l)] + (1-beta)*grads["db" + str(l)]
        # step along the smooth direction instead of the raw gradient
        parameters["w" + str(l)] -= learning_rate * v["dw" + str(l)]
        parameters["b" + str(l)] -= learning_rate * v["db" + str(l)]
    return parameters, v
```

KEY CONCEPT: ONE EXTRA KNOB, BETA

Momentum adds a single hyperparameter, beta , on top of the learning rate. The standard default is $\text{beta} = 0.9$ — averaging over roughly the last ten gradients — and it rarely needs changing. Note the velocity starts at zero, so the very first step is just $(1 - \text{beta})$ times the gradient, a small move that grows as history accumulates.

4. Adam: Momentum and RMSprop, Combined

Adam — short for **Adaptive Moment Estimation** — is the workhorse optimizer of modern deep learning. It keeps *two* running averages instead of one. The first, v , is the average of the gradients (this is Momentum). The second, s , is the average of the *squared* gradients (this is the RMSprop idea). It then uses s to scale each parameter's step individually: parameters whose gradients have been large take smaller, more careful steps, and those whose gradients have been small take larger ones.

Both averages are set up as zeros, exactly like the velocity but doubled:

```
def initialize_adam(parameters):
    L = len(parameters) // 2
    v, s = {}, {}
    for l in range(1, L + 1):
        v["dw" + str(l)] = np.zeros(parameters["W" + str(l)].shape)
        v["db" + str(l)] = np.zeros(parameters["b" + str(l)].shape)
        s["dw" + str(l)] = np.zeros(parameters["W" + str(l)].shape)
        s["db" + str(l)] = np.zeros(parameters["b" + str(l)].shape)
    return v, s
```

KEY CONCEPT: BIAS CORRECTION

Because v and s start at zero, their early values are biased toward zero — too small to trust for the first several steps. Adam corrects this by dividing each by $1 - \text{beta}^{**t}$, where t is the step number. Early on this divisor is well below 1, so it inflates the estimate back up; as t grows, beta^{**t} shrinks toward zero, the divisor approaches 1, and the correction quietly fades away.

The full update assembles all five pieces in order — momentum average, its correction, squared-gradient average, its correction, and the combined step:

```
def update_parameters_with_adam(parameters, grads, v, s, t,
                                learning_rate=0.01, beta1=0.9, beta2=0.999, epsilon=1e-8):
    L = len(parameters) // 2
    v_corrected, s_corrected = {}, {}
    for l in range(1, L + 1):
        # 1. momentum-style average of the gradient
        v["dw"+str(l)] = beta1*v["dw"+str(l)] + (1-beta1)*grads["dw"+str(l)]
        v["db"+str(l)] = beta1*v["db"+str(l)] + (1-beta1)*grads["db"+str(l)]
        # 2. bias-correct it
        v_corrected["dw"+str(l)] = v["dw"+str(l)] / (1 - beta1**t)
        v_corrected["db"+str(l)] = v["db"+str(l)] / (1 - beta1**t)
        # 3. RMSprop-style average of the SQUARED gradient
        s["dw"+str(l)] = beta2*s["dw"+str(l)] + (1-
beta2)*(grads["dw"+str(l)]**2)
        s["db"+str(l)] = beta2*s["db"+str(l)] + (1-
beta2)*(grads["db"+str(l)]**2)
        # 4. bias-correct it
```

```

s_corrected["dw"+str(l)] = s["dw"+str(l)] / (1 - beta2**t)
s_corrected["db"+str(l)] = s["db"+str(l)] / (1 - beta2**t)
# 5. combined step: direction from v, per-parameter scale from s
parameters["w"+str(l)] -= learning_rate * v_corrected["dw"+str(l)] \
    / (np.sqrt(s_corrected["dw"+str(l)]) + epsilon)
parameters["b"+str(l)] -= learning_rate * v_corrected["db"+str(l)] \
    / (np.sqrt(s_corrected["db"+str(l)]) + epsilon)
return parameters, v, s, v_corrected, s_corrected

```

KEY CONCEPT: ADAM'S DEFAULTS

Adam introduces three hyperparameters beyond the learning rate, and they almost never need tuning: $\beta_1 = 0.9$ (for the gradient average), $\beta_2 = 0.999$ (for the squared-gradient average), and $\epsilon = 1e-8$ (a tiny number added inside the denominator to prevent division by zero). The one knob still worth tuning is the learning rate itself.

5. Learning Rate Decay: Big Steps Early, Careful Steps Late

A single fixed learning rate forces a compromise: large enough to make fast early progress, but then too large to settle precisely — so training ends up bouncing around the minimum instead of resting in it. Learning rate decay resolves the tension by shrinking the rate over time. You take bold steps at the start and progressively gentler ones as you near the answer.

The simplest schedule decays a little on every epoch:

```

def update_lr(learning_rate0, epoch_num, decay_rate):
    return learning_rate0 / (1 + decay_rate * epoch_num)

```

A common variant holds the rate steady within fixed intervals and drops it in discrete steps, using `floor` to create the staircase:

```

def schedule_lr_decay(learning_rate0, epoch_num, decay_rate,
    time_interval=1000):
    return learning_rate0 / (1 + decay_rate * np.floor(epoch_num /
    time_interval))

```

KEY CONCEPT: ALWAYS DECAY THE ORIGINAL RATE

Both formulas divide `learning_rate0`, the *original* rate — never the current one. This keeps the schedule a clean, reproducible function of the epoch number, rather than a compounding shrinkage that would depend on every step taken before it.

6. One Model, Three Optimizers

The honest way to compare optimizers is to hold everything else fixed: the same model, the same data, the same gradient. Only the update rule changes. The companion script does exactly this, training a small model on a deliberately stretched problem (two features on very different scales) and printing the cost at the first few epochs:

GD	(lr=0.01)	->	cost @ epoch 1 = 3.830	@ epoch 3 = 0.259	final = 0.0113
MOMENTUM	(lr=0.01)	->	cost @ epoch 1 = 3.459	@ epoch 3 = 0.138	final = 0.0097
ADAM	(lr=0.05)	->	cost @ epoch 1 = 2.457	@ epoch 3 = 0.020	final = 0.0116

Two lessons stand out. First, all three reach essentially the same answer in the end — on a problem this simple, no optimizer is dramatically better at the finish line. The real difference is in the *path*: comparing the cost at epoch 1 and epoch 3, Momentum descends faster than plain gradient descent by smoothing the ravine’s oscillations, and Adam (given a larger learning rate) descends faster still.

Second, and just as important: Adam needs that larger learning rate to shine. With a small rate on an easy, bowl-shaped problem it can actually be *slower* than the simpler methods. Its decisive advantage appears on the harder, non-convex networks you will meet in the capstone — which is precisely why the right habit is to test all three rather than assuming the fanciest one wins.

KEY CONCEPT: DECAY LETS A SIMPLER OPTIMIZER CATCH UP

Run plain gradient descent at a fixed learning rate large enough to bounce around the minimum, then run it again with that same rate decayed over time. In the companion script the fixed run settles at a cost of 0.0169 while the decayed run reaches 0.0097 — matching what Momentum and Adam achieved. Good tuning narrows the gap: a well-scheduled simple optimizer can rival a fancy one.

7. Putting It All Together

The progression is the whole story. Mini-batch gradient descent makes many small, noisy updates per epoch instead of one slow update over all the data. Momentum smooths those noisy gradients with an exponentially weighted average — one extra knob, β , defaulting to 0.9 — so the network heads more directly downhill. Adam keeps a second average of the squared gradients and uses it to give every parameter its own step size, with bias correction to stay honest in the early steps; its defaults rarely need touching. And learning rate decay, layered on top of any of them, shrinks the step over time so training settles cleanly into the minimum.

THE MAIN TAKEAWAYS

All three optimizers answer the same question — which way and how far to step — with progressively smarter use of the gradient's history. Mini-batch GD uses the current gradient; Momentum averages recent gradients to cancel noise; Adam also scales each step by recent gradient size. None is universally best, so you compare them on your own problem. This is exactly the experiment the Part IV capstone scales up: implementing the optimizer trio from scratch, training one identical network with each on the same dataset, comparing their cost curves and decision boundaries, and finally adding learning rate decay to show that good tuning can let a simpler optimizer reach the same destination.